

CORE CONCEPTS

Extend Claude Code

Understand when to use CLAUDE.md, Skills, subagents, hooks, MCP, and plugins.

Claude Code combines a model that reasons about your code with **built-in tools** for file operations, search, execution, and web access. The built-in tools cover most coding tasks. This guide covers the extension layer: features you add to customize what Claude knows, connect it to external services, and automate workflows.

❗ For how the core agentic loop works, see [How Claude Code works](#).

New to Claude Code? Start with **CLAUDE.md** for project conventions, then add other extensions **as specific triggers come up**.

Overview

Extensions plug into different parts of the agentic loop:

- **CLAUDE.md** adds persistent context Claude sees every session
- **Skills** add reusable knowledge and invocable workflows
- **Code intelligence** connects Claude to a language server for symbol-level navigation and live type errors
- **MCP** connects Claude to external services and tools
- **Subagents** run their own loops in isolated context, returning summaries
- **Agent teams** coordinate multiple independent sessions with shared tasks and peer-to-peer messaging
- **Hooks** fire on lifecycle events and can run a script, HTTP request, prompt, or subagent
- **Plugins** and **marketplaces** package and distribute these features

Skills are the most flexible extension. A skill is a markdown file containing knowledge, workflows, or instructions. You can invoke skills with a command like `/deploy`, or Claude can load them

automatically when relevant. Skills can run in your current conversation or in an isolated context via subagents.

Match features to your goal

Features range from always-on context that Claude sees every session, to on-demand capabilities you or Claude can invoke, to background automation that runs on specific events. The table below shows what’s available and when each one makes sense.

Feature	What it does	When to use it	Example
CLAUDE.md	Persistent context loaded every conversation	Project conventions, “always do X” rules	”Use pnpm, not npm. Run tests before committing.”
Skill	Instructions, knowledge, and workflows Claude can use	Reusable content, reference docs, repeatable tasks	<code>/deploy</code> runs your deployment checklist; API docs skill with endpoint patterns
Subagent	Isolated execution context that returns summarized results	Context isolation, parallel tasks, specialized workers	Research task that reads many files but returns only key findings
<u>Agent teams</u>	Coordinate multiple independent Claude Code sessions	Parallel research, new feature development, debugging with competing hypotheses	Spawn reviewers to check security, performance, and tests simultaneously
<u>Code intelligence</u>	Language-server navigation and diagnostics	Typed languages, large codebases where grep is slow or imprecise	Jump to a symbol’s definition instead of reading the whole file
MCP	Connect to external services	External data or actions	Query your database, post to Slack, control a browser
Hook	Script, HTTP request, prompt, or subagent triggered by events	Automation that must run on every matching event	Run ESLint after every file edit
<u>Artifact</u>	Publish session output as a private, interactive web page	Output you want to see or share visually rather than as terminal text	An incident timeline that updates as Claude investigates

Plugins are the packaging layer. A plugin bundles skills, hooks, subagents, and MCP servers into a single installable unit. Plugin skills are namespaced (like `/my-plugin:review`) so multiple plugins can coexist. Use plugins when you want to reuse the same setup across multiple repositories or distribute to others via a marketplace.

Build your setup over time

You don't need to configure everything up front. Each feature has a recognizable trigger, and most teams add them in roughly this order:

Trigger	Add
Claude gets a convention or command wrong twice	Add it to <u>CLAUDE.md</u>
You keep typing the same prompt to start a task	Save it as a user-invocable <u>skill</u>
You paste the same playbook or multi-step procedure into chat for the third time	Capture it as a <u>skill</u>
You keep copying data from a browser tab Claude can't see	Connect that system as an <u>MCP server</u>
Claude reads many files to find where a symbol is defined or used	Install a <u>code intelligence plugin</u> for your language
A side task floods your conversation with output you won't reference again	Route it through a <u>subagent</u>
You want something to happen every time without asking	Write a <u>hook</u>
A second repository needs the same setup	Package it as a <u>plugin</u>

The same triggers tell you when to update what you already have. A repeated mistake or a recurring review comment is a CLAUDE.md edit, not a one-off correction in chat. A workflow you keep tweaking by hand is a skill that needs another revision.

Compare similar features

Some features can seem similar. Here's how to tell them apart.

Skill vs Subagent

Skills and subagents solve different problems:

- **Skills** are reusable content you can load into any context
- **Subagents** are isolated workers that run separately from your main conversation

Aspect	Skill	Subagent
What it is	Reusable instructions, knowledge, or workflows	Isolated worker with its own context

Aspect	Skill	Subagent
Key benefit	Share content across contexts	Context isolation. Work happens separately, only summary returns
<u>Context window impact</u>	Adds to your main window	Uses a separate window with its own input and output tokens
Best for	Reference material, invocable workflows	Tasks that read many files, parallel work, specialized workers

Skills can be reference or action. Reference skills provide knowledge Claude uses throughout your session (like your API style guide). Action skills tell Claude to do something specific (like `/deploy` that runs your deployment workflow). **Use a subagent** when you need context isolation or when your context window is getting full. The subagent might read dozens of files or run extensive searches, but your main conversation only receives a summary. Since subagent work doesn't consume your main context, this is also useful when you don't need the intermediate work to remain visible. Custom subagents can have their own instructions and can preload skills. **They can combine.** A subagent can preload specific skills (`skills:` field). A skill can run in isolated context using `context: fork` . See **Skills** for details.

CLAUDE.md vs Skill

Both store instructions, but they load differently and serve different purposes.

Aspect	CLAUDE.md	Skill
Loads	Every session, automatically	On demand
Can include files	Yes, with <code>@path</code> imports	Yes, with <code>@path</code> imports
Can trigger workflows	No	Yes, with <code>/<name></code>
Best for	"Always do X" rules	Reference material, invocable workflows

Put it in CLAUDE.md if Claude should always know it: coding conventions, build commands, project structure, "never do X" rules. **Put it in a skill** if it's reference material Claude needs sometimes (API docs, style guides) or a workflow you trigger with `/<name>` (deploy, review, release). **Rule of thumb:** Keep CLAUDE.md under 200 lines. If it's growing, move reference content to skills or split into `.claude/rules/` files.

CLAUDE.md vs Rules vs Skills

All three store instructions, but they load differently:

Aspect	CLAUDE.md	.claude/rules/	Skill
Loads	Every session	Every session, or when matching files are opened	On demand, when invoked or relevant
Scope	Whole project	Can be scoped to file paths	Task-specific
Best for	Core conventions and build commands	Language-specific or directory-specific guidelines	Reference material, repeatable workflows

Use **CLAUDE.md** for instructions every session needs: build commands, test conventions, project architecture. Use **rules** to keep CLAUDE.md focused. Rules with `paths` **frontmatter** only load when Claude works with matching files, saving context. Use **skills** for content Claude only needs sometimes, like API documentation or a deployment checklist you trigger with `/<name>`.

Subagent vs Agent team

Both parallelize work, but they’re architecturally different:

- **Subagents** run inside your session and report results back to your main context
- **Agent teams** are independent Claude Code sessions that communicate with each other

Aspect	Subagent	Agent team
Context	Own context window; results return to the caller	Own context window; fully independent
Communication	Reports results back to the main agent only	Teammates message each other directly
Coordination	Main agent manages all work	Shared task list with self-coordination
Best for	Focused tasks where only the result matters	Complex work requiring discussion and collaboration
Token cost	Lower: results summarized back to main context	Higher: each teammate is a separate Claude instance

Use a **subagent** when you need a quick, focused worker: research a question, verify a claim, review a

file. The subagent does the work and returns a summary. Your main conversation stays clean.**Use an agent team** when teammates need to share findings, challenge each other, and coordinate independently. Agent teams are best for research with competing hypotheses, parallel code review, and new feature development where each teammate owns a separate piece.**Transition point:** If you're running parallel subagents but hitting context limits, or if your subagents need to communicate with each other, agent teams are the natural next step.

ⓘ Agent teams are experimental and disabled by default. See [agent teams](#) for setup and current limitations.

MCP vs Skill

MCP connects Claude to external services. Skills extend what Claude knows, including how to use those services effectively.

Aspect	MCP	Skill
What it is	Protocol for connecting to external services	Knowledge, workflows, and reference material
Provides	Tools and data access	Knowledge, workflows, reference material
Examples	Slack integration, database queries, browser control	Code review checklist, deploy workflow, API style guide

These solve different problems and work well together:**MCP** gives Claude purpose-built tools for an external system, with the connection and authentication handled by the server.**Skills** give Claude knowledge about how to use those tools effectively, plus workflows you can trigger with `/<name>` . A skill might include your team's database schema and query patterns, or a `/post-to-slack` workflow with your team's message formatting rules.Example: An MCP server connects Claude to your database. A skill teaches Claude your data model, common query patterns, and which tables to use for different tasks.

Hook vs Skill

A hook fires on a lifecycle event; a skill is loaded into context for Claude to apply.

Aspect	Hook	Skill
Runs	A shell command, HTTP request, LLM prompt, or subagent	Instructions Claude reads and follows
Triggered by	Lifecycle events such as <code>PostToolUse</code> or <code>SessionStart</code>	You typing <code>/<name></code> , or Claude matching the description to your task
Determinism	Always fires on its event; the trigger is guaranteed	Claude interprets the instructions; outcome can vary
Context cost	Zero unless the hook returns output	Description loads each session; full content loads when used
Best for	Linting after edits, blocking unsafe commands, logging, notifications	Workflows that need reasoning, reference material, multi-step tasks

Use a hook when the action must happen the same way every time and doesn't need Claude to think. For example: format on save, reject `rm -rf /` , post a Slack message when a session ends. **Use a skill** when Claude should decide how to apply the steps, or when the content is knowledge rather than a script. For example: a `/release` checklist, your API style guide, a debugging playbook. **Put guardrails in hooks.** An instruction like “never edit `.env` ” in CLAUDE.md or a skill is a request, not a guarantee. A `PreToolUse` hook that blocks the edit is enforcement. If a rule must hold every time, make it a hook rather than a prompt instruction. **Hook output lands in context.** A `PostToolUse` hook that runs your linter feeds results back as text Claude reads; a `/fix-lint` skill tells Claude how to resolve them.

When: Session start**What loads:** Full content of all CLAUDE.md files (managed, user, and project levels).**Inheritance:** Claude reads CLAUDE.md files from your working directory up to the root, and discovers nested ones in subdirectories as it accesses those files. See **How CLAUDE.md files load** for details.

 Keep CLAUDE.md under 200 lines. Move reference material to skills, which load on-demand.

Skills are extra capabilities in Claude's toolkit. They can be reference material (like an API style guide) or invocable workflows you trigger with `/<name>` (like `/deploy`). Claude Code includes **bundled skills** like `/code-review` , `/batch` , and `/debug` that work out of the box. You can also create your

own. Claude uses skills when appropriate, or you can invoke one directly.**When:** Depends on the skill's configuration. By default, descriptions load at session start and full content loads when used. For user-only skills (`disable-model-invocation: true`), nothing loads until you invoke them.**What loads:** For model-invocable skills, Claude sees names and descriptions in every request. When you invoke a skill with `/<name>` or Claude loads it automatically, the full content loads into your conversation.**How Claude chooses skills:** Claude matches your task against skill descriptions to decide which are relevant. If descriptions are vague or overlap, Claude may load the wrong skill or miss one that would help. To tell Claude to use a specific skill, invoke it with `/<name>` . Skills with `disable-model-invocation: true` are invisible to Claude until you invoke them.**Context cost:** Low until used. User-only skills have zero cost until invoked.**In subagents:** Skills work differently in subagents. Instead of on-demand loading, skills listed in the subagent's `skills` field are fully preloaded into its context at launch. Subagents can still discover and invoke unlisted project, user, and plugin skills through the Skill tool.



Use `disable-model-invocation: true` for skills with side effects. This saves context and ensures only you trigger them.

When: Session start.**What loads:** Tool names from connected servers. Full JSON schemas stay deferred until Claude needs a specific tool.**Context cost:** Tool search is on by default, so idle MCP tools consume minimal context.



Run `/mcp` to see connection status and token costs per server. Claude Code reconnects to remote servers automatically if they drop, and you can disconnect servers you're not actively using.

When: After file edits, and on demand when Claude navigates code.**What loads:** Type errors and warnings after each file edit. Definition, reference, and type information when Claude looks up a symbol.**Context cost:** Low. Symbol lookups often replace broad file reads, so net context use can go down.



The LSP tool is inactive until you install a code intelligence plugin for your language.

When: On demand, when you or Claude spawns one for a task. **What loads:** Fresh, isolated context containing:

- The agent's own system prompt, not the full Claude Code system prompt
- Full content of skills listed in the agent's `skills:` field
- CLAUDE.md and git status, except the built-in Explore and Plan agents omit both
- Whatever context the lead agent passes in the prompt

Context cost: Isolated from main session. Subagents don't inherit your conversation history or invoked skills.

💡 Use subagents for work that doesn't need your full conversation context. Their isolation prevents bloating your main session.

When: On trigger. Hooks fire at specific lifecycle events like tool execution, session boundaries, prompt submission, permission requests, and compaction. See [Hooks](#) for the full list. **What loads:** Nothing by default. Hooks execute outside the main conversation. **Context cost:** Zero, unless the hook returns output that gets added as messages to your conversation.

💡 Hooks are ideal for side effects (linting, logging) that don't need to affect Claude's context.

Understand how features layer

Features can be defined at multiple levels: user-wide, per-project, via plugins, or through managed policies. You can also nest CLAUDE.md files in subdirectories or place skills in specific packages of a monorepo. When the same feature exists at multiple levels, here's how they layer:

- **CLAUDE.md files** are additive: all levels contribute content to Claude's context simultaneously. Files from your working directory and above load at launch; subdirectories load as you work in them. When instructions conflict, Claude uses judgment to reconcile them, with more specific instructions typically taking precedence. See [how CLAUDE.md files load](#).
- **Skills and subagents** override by name: when the same name exists at multiple levels, one

definition wins based on priority (managed > user > project for skills; managed > CLI flag > project > user > plugin for subagents). Plugin skills are namespaced to avoid conflicts. See skill discovery and subagent scope.

- **MCP servers** override by name: local > project > user. See MCP scope.
- **Hooks** merge: all registered hooks fire for their matching events regardless of source. See hooks.

Combine features

Each extension solves a different problem: CLAUDE.md handles always-on context, skills handle on-demand knowledge and workflows, MCP handles external connections, subagents handle isolation, and hooks handle automation. Real setups combine them based on your workflow.

For example, you might use CLAUDE.md for project conventions, a skill for your deployment workflow, MCP to connect to your database, and a hook to run linting after every edit. Each feature handles what it's best at.

Pattern	How it works	Example
Skill + MCP	MCP provides the connection; a skill teaches Claude how to use it well	MCP connects to your database, a skill documents your schema and query patterns
Skill + Subagent	A skill spawns subagents for parallel work	<code>/audit</code> skill kicks off security, performance, and style subagents that work in isolated context
CLAUDE.md + Skills	CLAUDE.md holds always-on rules; skills hold reference material loaded on demand	CLAUDE.md says “follow our API conventions,” a skill contains the full API style guide
Hook + MCP	A hook triggers external actions through MCP	Post-edit hook sends a Slack notification when Claude modifies critical files

Understand context costs

Every feature you add consumes some of Claude’s context. Too much can fill up your context window, but it can also add noise that makes Claude less effective; skills may not trigger correctly, or Claude may lose track of your conventions. Understanding these trade-offs helps you build an effective setup. For an interactive view of how these features combine in a running session, see Explore the context window.

Context cost by feature

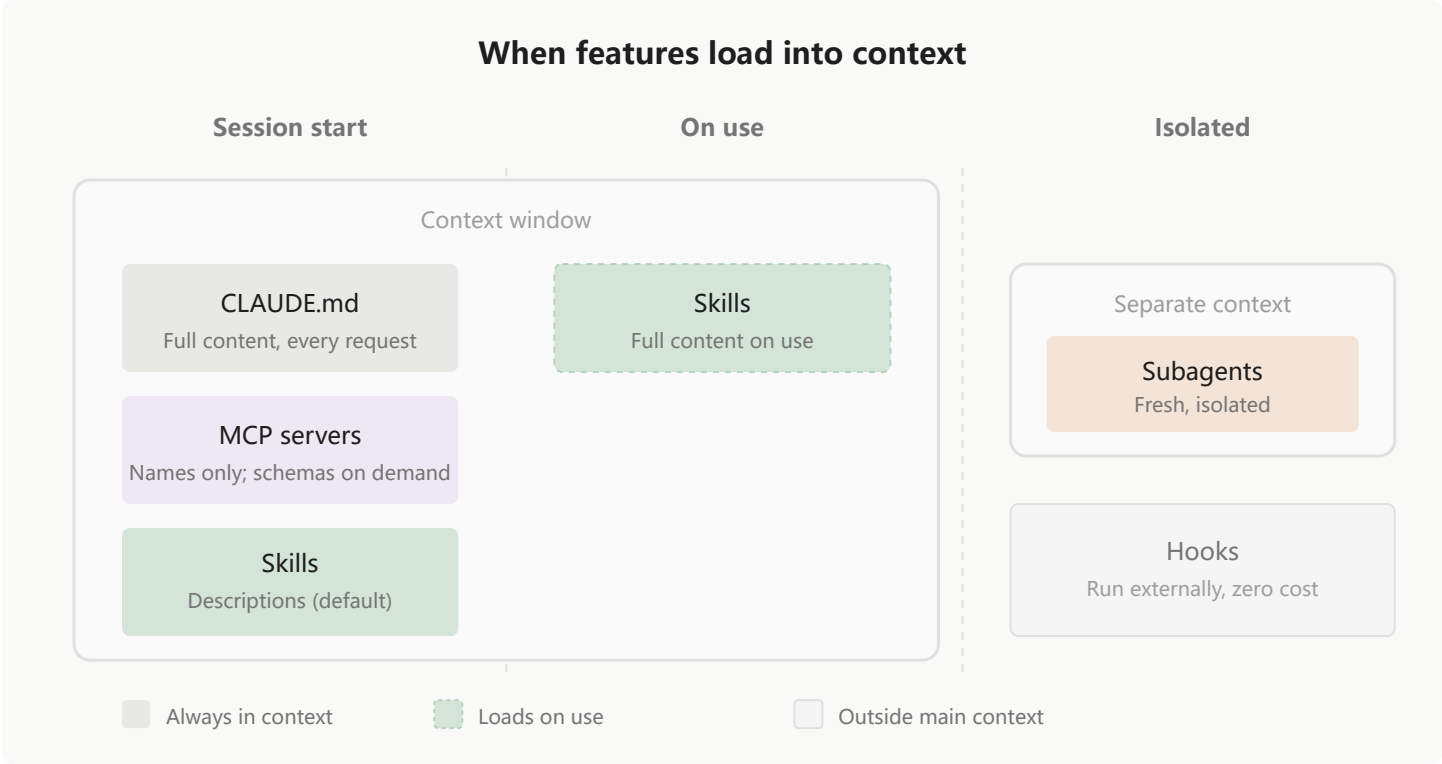
Each feature has a different loading strategy and context cost:

Feature	When it loads	What loads	Context cost
CLAUDE.md	Session start	Full content	Every request
Skills	Session start + when used	Descriptions at start, full content when used	Low (descriptions every request)*
MCP servers	Session start	Tool names; full schemas on demand	Low until a tool is used
Code intelligence	After file edits and on demand	Diagnostics after edits; symbol locations on lookup	Low; reduces file reads elsewhere
Subagents	When spawned	Fresh context with specified skills	Isolated from main session
Hooks	On trigger	Nothing (runs externally)	Zero, unless hook returns additional context

*By default, skill descriptions load at session start so Claude can decide when to use them. Set `disable-model-invocation: true` in a skill’s frontmatter to hide it from Claude entirely until you invoke it manually. This reduces context cost to zero for skills you only trigger yourself. For a skill you didn’t write, set `skill0verrides` in settings to do the same without editing its file.

Understand how features load

Each feature loads at different points in your session. The tabs below explain when each one loads and what goes into context.



CLAUDE.md

Skills and subagents solve different problems:

- **Skills** are reusable content you can load into any context
- **Subagents** are isolated workers that run separately from your main conversation

Aspect	Skill	Subagent
What it is	Reusable instructions, knowledge, or workflows	Isolated worker with its own context
Key benefit	Share content across contexts	Context isolation. Work happens separately, only summary returns
<u>Context window impact</u>	Adds to your main window	Uses a separate window with its own input and output tokens
Best for	Reference material, invocable workflows	Tasks that read many files, parallel work, specialized workers

Skills can be reference or action. Reference skills provide knowledge Claude uses throughout your session (like your API style guide). Action skills tell Claude to do something specific (like `/deploy` that runs your deployment workflow). **Use a subagent** when you need context isolation or when your context window is getting full. The subagent might read dozens of files or run extensive searches, but your main conversation only receives a summary. Since subagent work doesn't consume your main context, this is also useful when you don't need the intermediate work to remain visible. Custom

subagents can have their own instructions and can preload skills.**They can combine.** A subagent can preload specific skills (`skills:` field). A skill can run in isolated context using `context: fork` . See **Skills** for details.

Skills

Both store instructions, but they load differently and serve different purposes.

Aspect	CLAUDE.md	Skill
Loads	Every session, automatically	On demand
Can include files	Yes, with <code>@path</code> imports	Yes, with <code>@path</code> imports
Can trigger workflows	No	Yes, with <code>/<name></code>
Best for	"Always do X" rules	Reference material, invocable workflows

Put it in CLAUDE.md if Claude should always know it: coding conventions, build commands, project structure, “never do X” rules.**Put it in a skill** if it’s reference material Claude needs sometimes (API docs, style guides) or a workflow you trigger with `/<name>` (deploy, review, release).**Rule of thumb:** Keep CLAUDE.md under 200 lines. If it’s growing, move reference content to skills or split into `.claude/rules/` files.

MCP servers

All three store instructions, but they load differently:

Aspect	CLAUDE.md	<code>.claude/rules/</code>	Skill
Loads	Every session	Every session, or when matching files are opened	On demand, when invoked or relevant
Scope	Whole project	Can be scoped to file paths	Task-specific
Best for	Core conventions and build commands	Language-specific or directory-specific guidelines	Reference material, repeatable workflows

Use CLAUDE.md for instructions every session needs: build commands, test conventions, project architecture.**Use rules** to keep CLAUDE.md focused. Rules with `paths` **frontmatter** only load when Claude works with matching files, saving context.**Use skills** for content Claude only needs

sometimes, like API documentation or a deployment checklist you trigger with `<name>` .

Code intelligence

Both parallelize work, but they’re architecturally different:

- **Subagents** run inside your session and report results back to your main context
- **Agent teams** are independent Claude Code sessions that communicate with each other

Aspect	Subagent	Agent team
Context	Own context window; results return to the caller	Own context window; fully independent
Communication	Reports results back to the main agent only	Teammates message each other directly
Coordination	Main agent manages all work	Shared task list with self-coordination
Best for	Focused tasks where only the result matters	Complex work requiring discussion and collaboration
Token cost	Lower: results summarized back to main context	Higher: each teammate is a separate Claude instance

Use a subagent when you need a quick, focused worker: research a question, verify a claim, review a file. The subagent does the work and returns a summary. Your main conversation stays clean.**Use an agent team** when teammates need to share findings, challenge each other, and coordinate independently. Agent teams are best for research with competing hypotheses, parallel code review, and new feature development where each teammate owns a separate piece.**Transition point:** If you’re running parallel subagents but hitting context limits, or if your subagents need to communicate with each other, agent teams are the natural next step.

⚠ Agent teams are experimental and disabled by default. See [agent teams](#) for setup and current limitations.

Subagents

MCP connects Claude to external services. Skills extend what Claude knows, including how to use those services effectively.

Aspect	MCP	Skill
What it is	Protocol for connecting to external services	Knowledge, workflows, and reference material
Provides	Tools and data access	Knowledge, workflows, reference material
Examples	Slack integration, database queries, browser control	Code review checklist, deploy workflow, API style guide

These solve different problems and work well together:**MCP** gives Claude purpose-built tools for an external system, with the connection and authentication handled by the server.**Skills** give Claude knowledge about how to use those tools effectively, plus workflows you can trigger with `/<name>` . A skill might include your team’s database schema and query patterns, or a `/post-to-slack` workflow with your team’s message formatting rules.Example: An MCP server connects Claude to your database. A skill teaches Claude your data model, common query patterns, and which tables to use for different tasks.

Hooks

A hook fires on a lifecycle event; a skill is loaded into context for Claude to apply.

Aspect	Hook	Skill
Runs	A shell command, HTTP request, LLM prompt, or subagent	Instructions Claude reads and follows
Triggered by	Lifecycle events such as <code>PostToolUse</code> or <code>SessionStart</code>	You typing <code>/<name></code> , or Claude matching the description to your task
Determinism	Always fires on its event; the trigger is guaranteed	Claude interprets the instructions; outcome can vary
Context cost	Zero unless the hook returns output	Description loads each session; full content loads when used
Best for	Linting after edits, blocking unsafe commands, logging, notifications	Workflows that need reasoning, reference material, multi-step tasks

Use a hook when the action must happen the same way every time and doesn’t need Claude to think. For example: format on save, reject `rm -rf /` , post a Slack message when a session ends.**Use a skill** when Claude should decide how to apply the steps, or when the content is knowledge rather than a script. For example: a `/release` checklist, your API style guide, a debugging playbook.**Put guardrails in hooks.** An instruction like “never edit `.env` ” in CLAUDE.md or a skill is a request, not a guarantee.

A `PreToolUse` hook that blocks the edit is enforcement. If a rule must hold every time, make it a hook rather than a prompt instruction.**Hook output lands in context.** A `PostToolUse` hook that runs your linter feeds results back as text Claude reads; a `/fix-lint` skill tells Claude how to resolve them.

When: Session start**What loads:** Full content of all CLAUDE.md files (managed, user, and project levels).**Inheritance:** Claude reads CLAUDE.md files from your working directory up to the root, and discovers nested ones in subdirectories as it accesses those files. See [How CLAUDE.md files load](#) for details.

💡 Keep CLAUDE.md under 200 lines. Move reference material to skills, which load on-demand.

Skills are extra capabilities in Claude's toolkit. They can be reference material (like an API style guide) or invocable workflows you trigger with `/<name>` (like `/deploy`). Claude Code includes **bundled skills** like `/code-review`, `/batch`, and `/debug` that work out of the box. You can also create your own. Claude uses skills when appropriate, or you can invoke one directly.**When:** Depends on the skill's configuration. By default, descriptions load at session start and full content loads when used. For user-only skills (`disable-model-invocation: true`), nothing loads until you invoke them.**What loads:** For model-invocable skills, Claude sees names and descriptions in every request. When you invoke a skill with `/<name>` or Claude loads it automatically, the full content loads into your conversation.**How Claude chooses skills:** Claude matches your task against skill descriptions to decide which are relevant. If descriptions are vague or overlap, Claude may load the wrong skill or miss one that would help. To tell Claude to use a specific skill, invoke it with `/<name>`. Skills with `disable-model-invocation: true` are invisible to Claude until you invoke them.**Context cost:** Low until used. User-only skills have zero cost until invoked.**In subagents:** Skills work differently in subagents. Instead of on-demand loading, skills listed in the subagent's `skills` field are fully preloaded into its context at launch. Subagents can still discover and invoke unlisted project, user, and plugin skills through the Skill tool.

💡 Use `disable-model-invocation: true` for skills with side effects. This saves context and ensures only you trigger them.

When: Session start. **What loads:** Tool names from connected servers. Full JSON schemas stay deferred until Claude needs a specific tool. **Context cost:** Tool search is on by default, so idle MCP tools consume minimal context.

💡 Run `/mcp` to see connection status and token costs per server. Claude Code reconnects to remote servers automatically if they drop, and you can disconnect servers you're not actively using.

When: After file edits, and on demand when Claude navigates code. **What loads:** Type errors and warnings after each file edit. Definition, reference, and type information when Claude looks up a symbol. **Context cost:** Low. Symbol lookups often replace broad file reads, so net context use can go down.

💡 The LSP tool is inactive until you install a code intelligence plugin for your language.

When: On demand, when you or Claude spawns one for a task. **What loads:** Fresh, isolated context containing:

- The agent's own system prompt, not the full Claude Code system prompt
- Full content of skills listed in the agent's `skills:` field
- CLAUDE.md and git status, except the built-in Explore and Plan agents omit both
- Whatever context the lead agent passes in the prompt

Context cost: Isolated from main session. Subagents don't inherit your conversation history or invoked skills.

💡 Use subagents for work that doesn't need your full conversation context. Their isolation prevents bloating your main session.

When: On trigger. Hooks fire at specific lifecycle events like tool execution, session boundaries, prompt submission, permission requests, and compaction. See [Hooks](#) for the full list.**What loads:** Nothing by default. Hooks execute outside the main conversation.**Context cost:** Zero, unless the hook returns output that gets added as messages to your conversation.

💡 Hooks are ideal for side effects (linting, logging) that don't need to affect Claude's context.

Learn more

Each feature has its own guide with setup instructions, examples, and configuration options.



CLAUDE.md

Store project context, conventions, and instructions



Skills

Give Claude domain expertise and reusable workflows



Subagents

Offload work to isolated context



Agent teams

Coordinate multiple sessions working in parallel



MCP

Connect Claude to external services



Hooks

Automate actions with hooks



Plugins

Bundle and share feature sets



Marketplaces

Host and distribute plugin collections